

Protecciones Trazabilidad

Documento técnico integral del sistema de gestión y trazabilidad de relés de protección: arquitectura, modelo de dominio, flujo operativo, seguridad y estado actual del desarrollo.

Java 21 · Spring Boot 4

React 19 · TypeScript · Vite

PostgreSQL 16 · Flyway

Docker Compose

JWT · RBAC

Generado: 6 de julio de 2026

Rama actual: feature/frontend-react

Último commit: cd4967c — feat(auth): login JWT con roles
ADMIN/OPERADOR/AUDITOR, legajo y cambio de contraseña propio

Historial: 110 commits · 28 migraciones Flyway aplicadas (V1–V28)

Fuente: CLAUDE.md, README.md, docs/autenticacion.md y relevamiento directo del código fuente

Índice

1. Objetivo del sistema y contexto de negocio

2. Arquitectura general

Diagrama de capas · Stack tecnológico · Docker

3. Organización del código

Backend · Frontend

4. Modelo de datos

Entidades · Relaciones · Migraciones Flyway

5. Flujo de negocio y máquina de estados

Alta de relé · Movimientos · Baja lógica · Transiciones válidas

6. Autenticación y autorización (JWT + roles)

7. API REST — catálogo de endpoints

8. Reglas del proyecto y convenciones de código

9. Estado actual del desarrollo (historial de commits)

10. Riesgos, invariantes y próximos pasos

1. Objetivo del sistema

Qué problema resuelve y para quién.

Sistema web interno para **EPEC Transmisión** (Departamento de Teleoperaciones y Protecciones) que digitaliza la **trazabilidad e inventario de relés de protección** usados en estaciones transformadoras. Reemplaza un proceso manual que antes se llevaba en Microsoft Access.

EL SISTEMA NECESITA RESOLVER

- Saber qué **relé físico** (identificado por número de serie único) existe, en qué **estado operativo** está y en qué **posición/destino físico** se encuentra, en todo momento.
- Mantener un **historial inmutable** de todos los movimientos operativos que sufrió cada relé (auditoría).
- Controlar que los cambios de estado sigan un **flujo operativo válido** (máquina de estados), no cualquier transición arbitraria.
- Vincular cada relé con su documentación logística de origen: **remito** (proveedor) y/o **orden de provisión**, y llevar el control de garantía.
- Dar de baja relés preservando su historial (soft delete), nunca borrando datos operativos.
- Ofrecer un dashboard con KPIs operativos (stock, instalados, en reparación, garantías vencidas, documentación pendiente, etc.).

Concepto de dominio clave: **Modelo** = tipo técnico de relé (catálogo) — **Número de serie** = unidad física real (inventario). Un Modelo puede tener muchos Relés asociados. La trazabilidad y la operación siempre se hacen sobre la unidad física (el Relé), nunca sobre el Modelo.

Visión a futuro (README): el sistema busca servir como base para integraciones corporativas futuras — IBM Maximo, MIF, APIs REST, dashboards operativos, reporting técnico y auditoría operacional. Estos ítems son aspiracionales, no implementados hoy.

2. Arquitectura general

Tres capas físicas desacopladas, comunicadas exclusivamente por HTTP/REST.

```
React (SPA, puerto 5173, Vite)
  |   Axios → HTTP/JSON
  ▼
Spring Boot REST API (puerto 8080, prefijo /api)
  Controller → Service → Repository → Hibernate
  ▼
PostgreSQL (Docker, puerto host configurable, contenedor "protecciones-db")
```

- El frontend **no accede nunca directamente a la base de datos**; solo consume la API REST vía el cliente Axios centralizado (`frontend/src/api/axios.ts`).
- El backend expone **DTOs**, nunca entidades JPA, en sus respuestas.
- CORS permite cualquier puerto de `localhost / 127.0.0.1` (`allowedOriginPatterns` + `allowCredentials(true)` en `CorsConfig.java`), para que funcione tanto `npm run dev` (5173) como el build dockerizado (5173 vía Nginx).
- El esquema de base de datos es propiedad exclusiva de **Flyway**. Hibernate corre en modo `validate` (`spring.jpa.hibernate.ddl-auto=validate`): no crea ni modifica tablas, solo valida que las entidades coincidan con el esquema ya migrado.
- Autenticación con **JWT stateless** (`Authorization: Bearer <token>` , sin cookies ni sesión de servidor). Autorización centralizada en `SecurityConfig` (no `@PreAuthorize` disperso).
- Documentación de API autogenerada con **springdoc-openapi** (Swagger UI en `/swagger-ui/index.html`).

2.1 Stack tecnológico

BACKEND

Java 21 Spring Boot 4.0.6

Spring Data JPA / Hibernate Bean Validation

PostgreSQL driver Flyway Maven (mvnw)

springdoc-openapi Apache POI (Excel)

Lombok está en el `pom.xml` pero **no se usa** — getters/setters/construtores escritos a mano en todas las entidades/DTOs.

FRONTEND

React 19.2.6 TypeScript 6.0.2 Vite 8.0.12

Material UI (MUI v9) + emotion Axios 1.16.1

React Router DOM 7.15.1 Recharts 2.15.4

ESLint + typescript-eslint

2.2 Infraestructura Docker

- **Docker Compose** (`docker/docker-compose.yml`) levanta PostgreSQL (`postgres:16`), backend y frontend, cada uno en su contenedor, resueltos entre sí por nombre de servicio (el backend usa `postgres:5432` , no `localhost`).

- `backend/Dockerfile` : build multi-stage (`maven:3.9-eclipse-temurin-21` compila → `eclipse-temurin:21-jre-jammy` ejecuta). Expone 8080. `uploads/` se monta como volumen nombrado (`uploads_data`) para no perder PDFs adjuntos al recrear el contenedor.
- `frontend/Dockerfile` : build multi-stage (`node:22-alpine` corre `npm run build:docker` con `frontend/.env.docker`) → `nginx:1.27-alpine` sirve el `dist/` con fallback SPA (`try_files ... /index.html`) requerido por React Router.
- El frontend dockerizado se expone en el mismo puerto que `npm run dev` (`5173:80`), por lo que `CorsConfig.java` no necesita ajustes entre entornos.
- Variables de entorno DB: `DB_NAME` , `DB_USER` , `DB_PASSWORD` , `DB_PORT` (compose) y `DB_URL` / `SERVER_PORT` / `DDL_AUTO` / `TIMEZONE` / `JPA_SHOW_SQL` (Spring).

3. Organización del código

Estructura real de paquetes y carpetas, relevada directamente del repositorio.

3.1 Backend — `backend/src/main/java/protecciones/`

```
protecciones/
├── ProteccionesApplication.java      # entry point Spring Boot
├── config/
│   ├── CorsConfig.java             # política CORS
│   ├── SecurityConfig.java         # filter chain, reglas por rol, handlers 401/403
│   └── OpenApiConfig.java          # metadata de Swagger
├── security/                       # JwtService, JwtAuthenticationFilter,
│                                   # UserDetailsServiceImpl, CurrentUserProvider
├── controller/ (14 controllers)     # AuthController, DashboardController, DestinoController,
│                                   # EstadoController, LocalidadController, MarcaController,
│                                   # ModeloController, MovimientoController,
│                                   # OrdenProvisionController, PosicionController,
│                                   # ProveedorController, ProvinciaController,
│                                   # ReleController, RemitoController, UsuarioController
├── service/ (16 services)           # un service por recurso + AuthService + ReleBajaService
├── repository/                     # interfaces Spring Data JPA
├── entity/ (15 entidades)           # @Entity mapeadas 1:1 a tablas
├── dto/                            # *RequestDTO / *ResponseDTO
│   ├── auth/                      # LoginRequestDTO, LoginResponseDTO
│   └── dashboard/                 # DTOs específicos del dashboard
└── exception/                     # BusinessException, ErrorResponse, GlobalExceptionHandler
```

Migraciones SQL versionadas en `backend/src/main/resources/db/migration/V{n}__descripcion.sql`. Al momento de este relevamiento van de **V1 a V28** (V27 agrega autenticación, V28 agrega rol OPERADOR y número de sobre obligatorio).

3.2 Frontend — `frontend/src/`

```
src/
├── api/axios.ts                   # instancia única de Axios (baseUrl por env, interceptores de request/response)
├── context/AuthContext.tsx        # sesión (token/usuario), login()/logout(), isAdmin, canWrite
├── utils/authStorage.ts           # persistencia de token/usuario en localStorage
├── services/                      # una función por endpoint, agrupadas por recurso (*Service.ts)
├── types/                         # interfaces TS: entidad (Rele.ts) y su *Request (ReleRequest.ts)
├── components/
│   ├── admin/<recurso>/          # Form + Table por catálogo administrable (incluye admin/usuario)
│   ├── movimiento/              # MovimientoForm, MovimientoTable
│   ├── rele/                    # ReleForm, ReleTable
│   └── common/                  # EmptyState, PageHeader
├── pages/                        # HomePage, LoginPage, MovimientoPage, ReleDetailPage, RelePage
│   └── admin/                   # Destino, Localidad, Marca, Modelo, OrdenProvision, Posicion,
│                                   # Proveedor, Provincia, Remito, Usuario
└── layouts/MainLayout.tsx        # AppBar/navegación + <Outlet/>
```

— routes/	# AppRouter.tsx (rutas) + ProtectedRoute.tsx (guard de sesión)
— theme/theme.ts	# theme de Material UI (incluye modo oscuro persistente)



4. Modelo de datos

Entidades, relaciones y evolución del esquema vía Flyway.

4.1 Catálogos base

Entidad	Descripción
Marca	Fabricante del relé.
Estado	Estado operativo posible (<code>nombre</code> , <code>descripcion</code>).
Provincia → Localidad	Jerarquía geográfica (1:N).
Proveedor	Proveedor logístico.

4.2 Dominio principal

Entidad	Campos clave
Modelo	<code>marca_id</code> . Catálogo técnico (nombre + marca). Hasta V25 tenía también <code>tipo_id</code> y rango de tensión propios — eliminados por decisión de negocio ; no reintroducir sin migración explícita.
Rele	<code>modelo_id</code> , <code>remito_id?</code> , <code>orden_provision_id?</code> , <code>numeroSerie</code> (único, MAYÚSCULAS+trim), <code>codigoConfiguracion</code> (opcional, normalizado), <code>garantiaMeses</code> / <code>inicioGarantia</code> / <code>finGarantia</code> , <code>activo</code> + <code>motivoBaja</code> + <code>fechaBaja</code> (soft delete), <code>tipoIngreso</code> ("NUEVO" "USADO").
Movimiento	<code>rele_id</code> , <code>estado_id</code> , <code>posicion_id</code> , <code>usuario_id</code> . Evento histórico inmutable — nunca se actualiza ni se borra. Fuente de verdad del estado y posición actuales (= del último movimiento por fecha/id desc).
TransicionEstado	<code>estado_origen_id</code> , <code>estado_destino_id</code> . Define qué transiciones de estado están permitidas — la máquina de estados del sistema, modelada como datos.

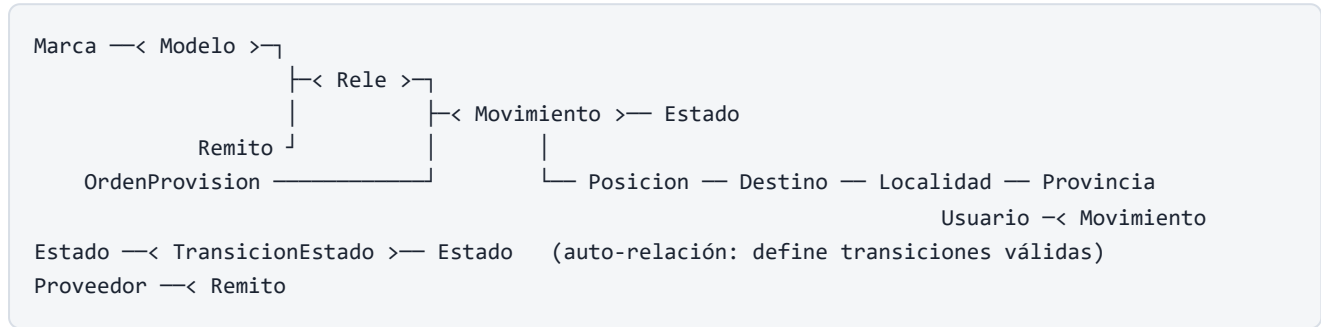
4.3 Ubicación física y logística

Entidad	Descripción
Destino (<code>localidad_id</code>)	Estación transformadora, laboratorio, depósito, etc.
Posicion (<code>destino_id</code>)	Ubicación puntual dentro de un Destino (banco de ensayo, tablero).
Remito (<code>proveedor_id</code>)	Comprobante de entrega, PDF adjunto opcional (<code>nombreArchivo</code> , <code>rutaArchivo</code>), flag <code>asociado</code> .
OrdenProvision	Orden de compra/provisión; también admite PDF adjunto y flag <code>asociado</code> .

4.4 Usuarios

Usuario: nombre, apellido, email, `numeroSobre` (legajo interno, `NOT NULL UNIQUE` igual que `Rele.numeroSerie`, alternativa de login al email), `passwordHash` (BCrypt, `NULL` para cuentas que no pueden loguearse), `rol` (`"ADMIN" | "OPERADOR" | "AUDITOR"`, string validado en `UsuarioService` + `CHECK` en BD, sin enum Java), `activo` (deshabilita login preservando historial). Es también el "responsable" de cada Movimiento. El id 1 (`sistema@local`) es una cuenta histórica sin `passwordHash` — no puede loguearse.

4.5 Diagrama de relaciones



4.6 Evolución del esquema (Flyway, últimas migraciones)

Versión	Descripción
V15	Refactor de estados operativos
V16 / V16_1	Flag <code>asociado</code> en documentos + seed de tipos faltantes
V17 / V18 / V19	Importación masiva de modelos y relés (datos reales)
V20	Actualización de transiciones de estado (máquina de estados vigente)
V21	Agrega <code>codigoConfiguracion</code> a Rele
V22 / V23	Remueve y luego reintroduce <code>asociado</code> en documentos
V24	Agrega <code>tipoIngreso</code> a Rele
V25	Limpieza de estructura Modelo/Rele — elimina <code>tipo_id</code> y tensión
V26	Crea vista <code>ultimo_movimiento</code>
V27	Agrega autenticación a <code>usuario</code> (JWT, rol, password_hash)
V28	Agrega rol <code>OPERADOR</code> y <code>numero_sobre</code> (legajo, único, obligatorio)

⚠ Existen nombres de estado de migraciones anteriores (`INSTALADO`, `DISPONIBLE`, `REPARACION`, `BANCO ENSAYO`, etc.) que ya no forman parte del flujo vigente en `transicion_estado`, pero pueden seguir existiendo como filas en la tabla `estado` o referenciados en datos históricos. Antes de asumir el nombre exacto de un estado, verificar el contenido real de la tabla.

5. Flujo de negocio y máquina de estados

Cómo se crea, mueve y da de baja un relé, paso a paso.

5.1 Alta de un relé (`ReleService.guardar`)

1. Se valida que exista el `Modelo` referenciado.
2. Se resuelven opcionalmente `Remito` y/o `OrdenProvision` si vienen en el DTO.
3. Se valida que **no exista otro relé con el mismo `numeroSerie`** → si existe, `BusinessException` .
4. Se normalizan `numeroSerie` y `codigoConfiguracion` (mayúsculas/trim).
5. Si `cargarGarantia = true` , se calculan `inicioGarantia` (hoy si no se especifica) y `finGarantia` (inicio + meses); si no, quedan en `null` .
6. Se guarda el `Rele` con `activo = true` .
7. **Se crea automáticamente el primer `Movimiento`** : estado `"EN STOCK"` , posición inicial del DTO, usuario = sistema, nota fija "Ingreso inicial del relé". Un relé sin movimientos es un estado inconsistente.

5.2 Movimiento operativo (`MovimientoService.guardar`)

1. Se valida que el relé exista y esté **activo**.
2. Se busca el **último movimiento** del relé (fecha desc, id desc como desempate).
3. Si existe un movimiento previo, se valida la transición `estadoActual → estadoDestino` contra `TransicionEstado` . Si no está permitida → `BusinessException` .
4. Se inserta el nuevo `Movimiento` (nunca se modifica uno anterior).
5. **Caso especial**: si el estado destino es `"BAJA"` , se dispara automáticamente `ReleBajaService.aplicarBaja()` .

5.3 Baja de un relé — dos caminos

A) TRANSICIÓN DE ESTADO NORMAL

Registrar un `Movimiento` con estado `"BAJA"` vía `MovimientoService.guardar` , sujeto a que la transición esté permitida.

B) ENDPOINT DEDICADO

`PATCH` `/api/relés/{id}/baja`

Verifica activo → exige historial previo → valida transición → inserta `Movimiento` a `BAJA` → llama `ReleBajaService.aplicarBaja` .

En ambos caminos: `activo = false` , `fechaBaja = now()` , `motivoBaja = motivo` . Es **soft delete** — el historial permanece intacto.

5.4 Máquina de estados vigente (V20)

EN STOCK	→ ENSAYO, GARANTIA_PROVEEDOR, APROBADO, RESERVA, BAJA
ENSAYO	→ APROBADO, GARANTIA_PROVEEDOR, BAJA
GARANTIA_PROVEEDOR	→ APROBADO, BAJA
APROBADO	→ EN SERVICIO, RESERVA, BAJA
RESERVA	→ EN SERVICIO, BAJA

EN SERVICIO → EN REPARACION, BAJA
EN REPARACION → EN STOCK, BAJA

EN STOCK es el estado de entrada (asignado al alta) y BAJA es terminal — una vez ahí, el relé queda inactivo y no admite más movimientos.

6. Autenticación y autorización

JWT stateless con 3 roles, implementado en V27 y ampliado en V28.

6.1 Estrategia

- **JWT stateless** vía `Authorization: Bearer <token>`. Sin sesión ni cookies del servidor.
- Login por **email o número de sobre (legajo)** — ambos únicos.
- Token firmado con `JWT_SECRET`, expira a las `JWT_EXPIRATION_MS` (default: 8 horas). El subject del JWT siempre es el email, sin importar con qué identificador se logueó.
- Sin refresh token ni recuperación por email — cualquier usuario logueado puede cambiar su propia contraseña.

6.2 Roles

Rol	Lectura (GET)	Escritura catálogos/relés/mov.	Gestión de usuarios
ADMIN	Sí	Sí	Sí (único rol que puede)
OPERADOR	Sí	Sí	No
AUDITOR	Sí	No	No

Regla centralizada en `SecurityConfig` (no `@PreAuthorize` disperso): `PUT /api/auth/password` → cualquier rol autenticado · cualquier `GET /api/**` → los 3 roles · escritura en `/api/usuarios/**` → solo ADMIN · escritura en el resto → ADMIN u OPERADOR.

6.3 Modelo de datos de `usuario` (ampliado en V27/V28)

Columna	Notas
<code>password_hash</code>	BCrypt. <code>NULL</code> para "sistema" (id=1) — no puede loguearse, se conserva por integridad referencial.
<code>rol</code>	<code>'ADMIN' 'OPERADOR' 'AUDITOR'</code> , CHECK en BD + validado en <code>UsuarioService</code> .
<code>activo</code>	Deshabilita login sin romper historial de <code>Movimiento.usuario_id</code> .
<code>numero_sobre</code>	Legajo interno EPEC. NOT NULL UNIQUE. Usuarios preexistentes a V28 recibieron placeholder = su propio id — corregir desde <code>/admin/usuarios</code> .

6.4 Flujo de login

`identificador` acepta email o número de sobre — `UserDetailsServiceImpl` prueba primero por email, luego por `numeroSobre`. Si no matchea (inactivo, sin password, o password incorrecto), `AuthService.login` devuelve un `BusinessException` genérico ("Email o contraseña incorrectos") sin distinguir el motivo, para no filtrar qué cuentas existen.

6.5 Frontend

El interceptor de `axios.ts` agrega el header `Authorization` en cada llamada si hay token guardado; cualquier `401` fuerza logout + redirección a `/login`. `AuthContext` expone `isAdmin` (gatea `/admin/usuarios`) y `canWrite` (`ADMIN` u `OPERADOR`, gatea el resto de la escritura). La autoridad real siempre es el backend.

6.6 Credencial de bootstrap

Campo	Valor
Email	<code>admin@epec.local</code>
Número de sobre (placeholder)	<code>"2"</code>
Contraseña temporal	<code>EpecAdmin2026!</code>

Cambiar esta contraseña y el legajo placeholder antes de cualquier uso real fuera de desarrollo local.

6.7 Limitaciones conocidas

- Sin refresh token: la sesión expira a las 8h.
- Sin recuperación de contraseña "olvidé mi contraseña".
- Sin 2FA.
- Sin bloqueo por intentos fallidos de login.

7. API REST — catálogo de endpoints

Relevado directamente de las anotaciones `@RequestMapping` de cada controller (14 controllers).

7.1 Autenticación

Endpoint	Descripción
POST /api/auth/login	Login por email o número de sobre → JWT
GET /api/auth/me	Datos del usuario autenticado
PUT /api/auth/password	Autogestión de la propia contraseña

7.2 Relés

Endpoint	Descripción
GET /api/relés?page&size&sort	Listado paginado + sorting dinámico
GET /api/relés/serial/{numeroSerie}	Búsqueda exacta por serial
GET /api/relés/buscar?serial=	Búsqueda parcial
GET /api/relés/{id}	Detalle de relé
GET /api/relés/{id}/movimientos	Historial completo del relé
GET /api/relés/{id}/estado-actual	Estado derivado del último movimiento
GET /api/relés/estado/{estado}	Filtrar relés por estado actual
GET /api/relés/opciones	Opciones para selects del frontend
POST /api/relés	Alta de relé (+ movimiento inicial EN STOCK)
PUT /api/relés/{id}	Edición de relé
PATCH /api/relés/{id}/baja	Baja lógica dedicada

7.3 Movimientos

Endpoint	Descripción
GET /api/movimientos	Listado de movimientos
POST /api/movimientos	Registrar movimiento operativo (valida transición)
GET /api/movimientos/exportar	Export Excel filtrado del historial

7.4 Dashboard

Endpoint	Descripción
GET /api/dashboard/kpis	KPIs generales (stock, instalados, garantías, etc.)

GET /api/dashboard/movimientos	Últimos movimientos
GET /api/dashboard/marcas	Distribución por marca
GET /api/dashboard/modelos	Distribución por modelo
GET /api/dashboard/estados	Distribución por estado
GET /api/dashboard/destinos	Distribución por destino
GET /api/dashboard/proveedores	Distribución por proveedor
GET /api/dashboard/usuarios	Distribución por usuario responsable
GET /api/dashboard/exportar	Export Excel de KPIs
GET /api/dashboard/exportar-pdf	Export PDF de KPIs

7.5 Catálogos y logística

Todos siguen el mismo patrón CRUD (**GET** lista / **GET** por relación / **POST** / **PUT** /{id} / **DELETE** /{id}), con variantes documentales para Remito/OrdenProvision.

Recurso	Base	Extras
Marca	/api/marcas	—
Modelo	/api/modelos	—
Estado	/api/estados	GET /transiciones/{releId} (próximos estados válidos)
Provincia	/api/provincias	—
Localidad	/api/localidades	GET /provincia/{provinciaId}
Destino	/api/destinos	GET /localidad/{localidadId}
Posicion	/api/posiciones	GET /destino/{destinoId}
Proveedor	/api/proveedores	—
Remito	/api/remitos	Upload/descarga de PDF (/id/archivo), /disponibles
OrdenProvision	/api/ordenes-provision	Upload/descarga de PDF (/id/archivo), /disponibles
Usuario	/api/usuarios	Solo ADMIN en escritura; sin DELETE (se desactiva)

8. Reglas del proyecto y convenciones

Invariantes de diseño implícitas en el código existente, a respetar en todo cambio nuevo.

8.1 Reglas de arquitectura y negocio

1. Nunca acceder a la base de datos desde un Controller — solo llaman a un Service, inyectado por constructor.
2. Toda la lógica de negocio vive en la capa Service, nunca en Controller o Repository.
3. Los Controllers solo coordinan request/response: reciben DTOs, delegan, devuelven DTOs. Sin `if` de negocio.
4. Se exponen siempre DTOs, nunca entidades JPA. Mappers `entity → DTO` viven dentro del Service.
5. Todo cambio de esquema se hace exclusivamente vía migración Flyway nueva. Hibernate en modo `validate`.
6. Separación estricta frontend/backend — sin SSR, comunicación solo por HTTP/JSON vía `axios.ts`.
7. Los movimientos son inmutables: se insertan, nunca se actualizan ni eliminan. Estado/posición actual siempre derivados del último movimiento.
8. La baja de un relé es lógica (soft delete): `activo=false` + `motivoBaja` + `fechaBaja`. Nunca se borra el registro ni su historial.
9. Las transiciones de estado se validan siempre contra `transicion_estado`, nunca hardcodeadas en Java.
10. Números de serie son únicos y se normalizan (`trim().toUpperCase()`); la unicidad se valida explícitamente en el Service.
11. 3 roles JWT (ADMIN/OPERADOR/AUDITOR). El usuario autenticado se obtiene siempre vía `CurrentUserProvider.obtenerUsuarioActual()`, nunca hardcodeado.
12. Errores de negocio se señalizan con `BusinessException` (→ HTTP 400), no con excepciones genéricas ni `null` silencioso.
13. Archivos adjuntos (Remito, OrdenProvision) se guardan en filesystem local (`uploads/...`) con nombre prefijado por timestamp; la ruta se persiste en la entidad.

8.2 Convenciones de código — Backend (Java)

- Constructor injection siempre (nunca `@Autowired` en campo).
- Nombres de métodos/variables en **español** (`guardar`, `obtenerTodos`, `darDeBaja`, `ultimoMovimiento`).
- Estilo de formato muy vertical/expandido: un argumento o expresión por línea — es el estilo dominante del repo, no "corregir" sin pedido explícito.
- Getters/setters explícitos escritos a mano (sin Lombok, pese a estar en `pom.xml`).
- Repositorios: se prioriza derived query methods, y se cae a `@Query` JPQL (text block) solo cuando el derived method no alcanza.
- Nombres de tabla/columna en `snake_case`; nombres de campo Java en `camelCase`.

8.3 Convenciones de código — Frontend (TypeScript/React)

- Un archivo de tipo por entidad (`types/Rele.ts`) + su contraparte de request cuando difiere.
- Un archivo de service por recurso, funciones sueltas (no clases) que envuelven `api.get/post/put/patch` .
- Componentes divididos en Form + Table por recurso, bajo `components/<recurso>/` .
- Layout de rutas centralizado en `routes/AppRouter.tsx` , todo bajo un único `MainLayout` .
- Mismo estilo vertical/expandido que el backend: imports, JSX props y llamadas partidas en muchas líneas cortas.
- Material UI como única librería de componentes visuales — no mezclar con otra.

9. Estado actual del desarrollo

110 commits acumulados. Línea de tiempo funcional reciente (más nuevo primero).

Commit	Descripción
cd4967c	feat(auth): login JWT con roles ADMIN/OPERADOR/AUDITOR, legajo y cambio de contraseña propio
f51f9b0	feat/dashboard): KPIs dinámicos por estado/destino/proveedor/usuario, export Excel/PDF y filtros
09e911c	feat(movimientos): navegación cruzada entre movimientos y detalle de relé
7935e57	feat(theme): modo oscuro con toggle persistente
0bdd419	feat(relés): filtros combinados, paginación configurable y página de detalle
5602dec	fix(relés): validar posición inicial requerida al crear un relé
6e67c82	feat(relés): carga en lote, detección de duplicados y mejoras de UX
8f2dfb6	feat(movimientos): confirmar antes de dar de baja un relé
78aea89	feat(layout): AppBar responsive con menú lateral en pantallas chicas
925f381	feat/dashboard): gráfico de distribución de relés por modelo
eb78cb4	feat: carga y visualización de PDF en remitos y órdenes de provisión
cc0fbb5	refactor: eliminar Tipo y atributos de tensión del modelo de relés
b30c3d1	feat: dockerizar backend y frontend completos
a28deee	feat/dashboard): métricas operativas y monitoreo inicial

El resto del historial (~96 commits anteriores) cubre la construcción incremental de los CRUDs base, exportación Excel de movimientos, gestión de garantía, código de configuración y el módulo de reporting inicial del dashboard.

9.1 Funcionalidades implementadas (consolidado)

BACKEND

- 14 controllers / 16 services / 15 entidades
- 28 migraciones Flyway (V1–V28)
- Autenticación JWT + RBAC de 3 roles
- Máquina de estados validada por datos
- Export Excel (Apache POI) y PDF de KPIs/movimientos
- Upload/descarga de PDFs adjuntos (Remito/Orden)
- Swagger UI autogenerado

FRONTEND

- Login + guard de sesión + gestión de roles en UI
- Dashboard con KPIs dinámicos y gráficos (Recharts)
- Filtros combinados, paginación configurable, sorting
- Carga en lote de relés con detección de duplicados
- Navegación cruzada relé ↔ movimiento
- Modo oscuro persistente
- Layout responsive (AppBar + menú lateral)

10. Invariantes, riesgos y próximos pasos

Lo que nunca debe romperse, y lo que queda pendiente.

10.1 Cosas que nunca deben romperse

- **Inmutabilidad de `Movimiento`** : es el activo más importante del sistema (la razón de ser del proyecto es la trazabilidad).
- **Estado/posición como derivados** del último movimiento — no introducir campos persistidos que puedan desincronizarse.
- **Validación de transiciones contra `transicion_estado`** — no hardcodear reglas en Java ni saltarlas en un endpoint nuevo.
- **Unicidad de `numeroSerie`** — revalidar siempre en el Service, con mensaje de negocio amigable.
- **Orden de migraciones Flyway** — nunca editar un script ya aplicado; nunca cambiar `ddl-auto` de `validate` .
- **Soft delete de `Rele`** — ningún flujo nuevo debe hacer `DELETE` físico de un relé con movimientos.
- **Separación DTO/Entity** — no devolver entidades JPA por comodidad.
- **Contrato de paginación** (`Page<ReleResponseDTO>`) — cambiarlo rompe todos los consumidores del frontend.

10.2 Deuda / limitaciones conocidas

- Sin refresh token, sin recuperación de contraseña, sin 2FA, sin bloqueo por intentos fallidos (ver sección 6.7).
- Usuarios preexistentes a V28 tienen un legajo *placeholder* (su propio id) pendiente de corrección manual.
- README.md raíz describe una versión anterior del dominio (menciona `Tipo` y tensiones auxiliares, eliminados en V25/commit `cc0fbb5`) — el `CLAUDE.md` es la fuente de verdad actualizada, no el README.

10.3 Próximos pasos sugeridos (visión README, no comprometidos)

- Integraciones corporativas: IBM Maximo, MIF, APIs externas.
- Auditoría automática más allá del historial de movimientos.
- Ampliación de reporting técnico y dashboards operativos.

Documento generado automáticamente a partir de CLAUDE.md, README.md, docs/autenticacion.md y relevamiento directo del código fuente del repositorio `protecciones-trazabilidad` , rama `feature/frontend-react` .